

WORKSHOP BLUEPRINT

Build the Agent, Not Just the Prompt

To get engineers **real, hands-on experience** with Agentic AI, skip heavy extensions like Cursor or Copilot. They abstract away the orchestration logic — the loops, state management, and tool calling your team needs to understand. Build it yourselves instead.

THE APPROACH

The Lean Agentic Harness

A minimal stack that exposes the **actual mechanics** of agentic systems — no black boxes, no hidden orchestration.



Python · FastAPI

Write raw agent loops, define custom tools, and manage state explicitly.



React Frontend

Lightweight UI to observe agent decisions and step through execution.

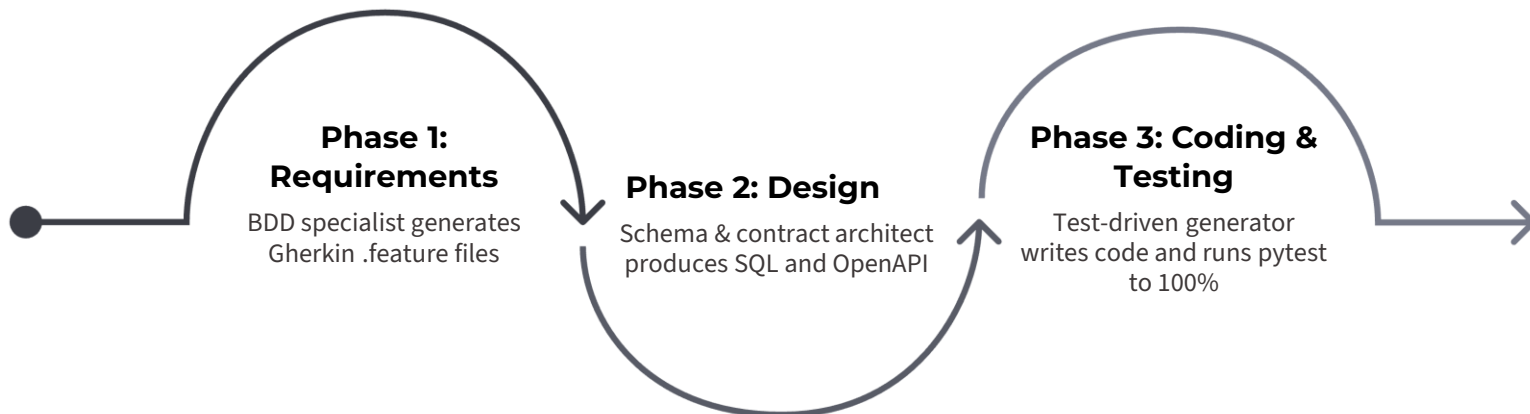


VS Code Sandbox

Agents read and write files directly in the local workspace — visible, inspectable, debuggable.

Three Agents. One SDLC. Real Learning.

Each proof-of-concept maps to a core phase of the Software Development Life Cycle — giving the team end-to-end exposure to agentic patterns in a realistic workflow.



Each phase introduces a distinct agentic pattern — from critic loops to tool routing to execution sandboxes.

The Requirements Agent — BDD Specialist

Goal

Turn vague natural-language feature requests into structured, unambiguous Gherkin scenarios (`.feature` files).

Pattern

Critic / Refinement Loop

How It Works

- **Agent 1 — Author:** Drafts initial user stories and acceptance criteria from the raw requirement.
- **Agent 2 — Verifier/Critic:** Acts as QA Lead, flagging ambiguity, missing edge cases, and untestable statements.
- Feedback loops back until a quality threshold is met, then the clean `.feature` file is written to the workspace.

The Design Agent — Schema & Contract Architect

Goal

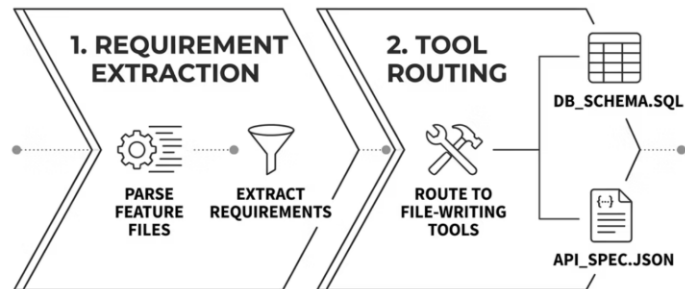
Read generated `.feature` files and automatically produce a technical design spec — database schemas and API contracts.

Pattern

Tool-Use (Routing)

Hands-On Learning Point

Your team learns to give an LLM a defined set of Python functions — `read_file()`, `write_file()` — that let it operate directly inside the local directory.



The Coding & Testing Agent — Test-Driven Generator

Goal

Write application code and unit tests, then **self-correct** until all tests pass.

Pattern

Human-in-the-Loop + Execution Sandbox

The Core Loop

01

Read `api_spec.json` and `db_schema.sql`, generate FastAPI boilerplate.

02

Generate `pytest` file from BDD scenarios.

03

Execute `pytest` via `run_command` tool in the local terminal.

04

Read traceback, patch source code, re-run — repeat until **100% pass rate**.

Self-Correcting Coding Agent: Write → Test → Fix Loop

A minimal, end-to-end implementation of an autonomous code-fix agent using **Python**, **FastAPI**, and the **OpenAI / Azure OpenAI** library. The agent runs a tight **Write** → **Test** → **Fix** loop, exposing two tools to the LLM: one to overwrite application code, and one to run `pytest` and return terminal output.

Write Code

Run Tests



Apply Fixes

Collect Failures

Each cycle appends fresh error traces to the message history, driving iterative correction until tests pass or the iteration cap is reached.

Prerequisites

- Install `openai` and `pytest` in your workspace
- Prepare a target file to fix — e.g. a broken `calc.py`
- Write a corresponding `test_calc.py` test suite

The Agent Loop (`agent.py`)

This script orchestrates the full cycle: it executes tests, evaluates output, and feeds failure logs back to the LLM. The loop terminates when all tests pass or the maximum iteration count is reached.

What Your Team Learns by Building This From Scratch

When your team implements this sequence themselves, they encounter foundational agentic-system concepts that are difficult to grasp from documentation alone.

The Tool Abstraction

The model *does not* execute Python or access the terminal. It outputs a structured JSON payload requesting an action. The local script acts as the "operating system" — interpreting and executing each command. This separation of **intent** and **execution** is the cornerstone of safe agentic design.

State Control Friction

If the model edits code but forgets to call `run_tests()`, the loop stalls. Teams learn how **prompt engineering** and stricter system control loops enforce behavioural pacing — ensuring the agent always signals the next step.

Context Window Expansion

Each execution cycle appends new error traces to the `messages` array, making token consumption visible in real time. Teams observe how conversational memory grows across extended debugging runs — and why **truncation strategies** and **summary compression** become essential at scale.